

Case 1: Campus Lost & Found Portal

FindIt — Smart Campus Lost & Found System

Your Name

May 12, 2026

Problem Statement

Objective

Replace traditional campus notice-board style lost-and-found workflows with a modern digital platform.

Problems with Existing System

- Manual paper notices
- Difficult item discovery
- No centralized tracking
- Slow claim verification
- Poor accessibility for students

Goal

Build a mobile-friendly platform that feels modern, intuitive, and easy to use.

Project Overview

Project Built

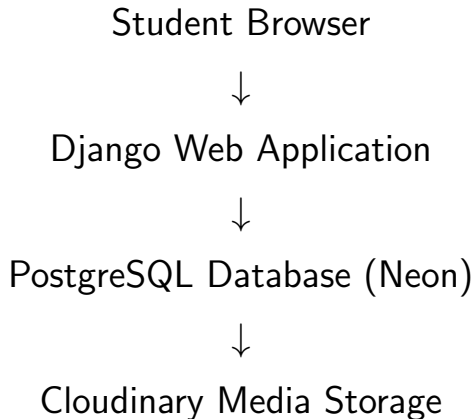
FindIt — Campus Lost & Found Portal

Core Features

- Post LOST items
- Post FOUND items
- Upload item images
- Search and filtering
- Smart item matching
- Claim request workflow
- User authentication

Design Philosophy

Instagram-inspired social feed instead of traditional admin-style portals.



Render handles deployment and automatic redeployment on Git push.

Technology Stack

Layer	Technology
Backend	Django 5.2
Database	PostgreSQL (Neon)
Frontend	Django Templates + Tailwind CSS
Authentication	Django Auth System
Media Storage	Cloudinary
Matching Engine	RapidFuzz
Deployment	Render
CI/CD	GitHub + Render Auto Deploy

Core Features

Lost & Found Posts

- Upload images
- Add title and description
- Select location and date
- Categorize items

Search & Discovery

- Search by keywords
- Filter by category
- Filter by lost/found type
- Location-based filtering

Claim Workflow

- Submit claim request
- Owner approval/rejection

Smart Matching System

Purpose

Automatically suggest possible LOST FOUND item matches.

Matching Strategy

- 40% → title similarity
- 30% → description similarity
- 20% → location similarity
- 10% → date proximity

Technology Used

RapidFuzz fuzzy matching library

Goal

Reduce manual searching effort for students.

Database Design

Main Models

- User
- Item
- Claim

Item Model Includes

- title
- description
- image
- item type (lost/found)
- location
- date
- owner

Claim Model Includes

Design Goals

- Mobile-first experience
- Minimal and modern interface
- Social-media-inspired feed
- Easy item discovery

UI Features

- Responsive card layout
- Large item images
- Clean typography
- Simple navigation
- Fast interaction flow

Reasoning

The platform should feel approachable and human-centered rather than institutional.

Deployment Setup

Hosting Stack

- Render → Django hosting
- Neon → PostgreSQL database
- Cloudinary → image hosting
- GitHub → version control

Production Environment Variables

```
DATABASE_URL=postgresql://...  
SECRET_KEY=...  
DEBUG=False
```

```
CLOUDINARY_CLOUD_NAME=...  
CLOUDINARY_API_KEY=...  
CLOUDINARY_API_SECRET=...
```

Security Features

- CSRF protection
- Authentication checks
- Protected claim workflow
- Secure image uploads
- Environment variable based secrets

Operational Features

- Automatic deployment
- Cloud-based media storage
- PostgreSQL persistence
- Production configuration separation

Key Learnings

- Full-stack Django development
- PostgreSQL integration
- Cloud deployment workflows
- Media storage handling
- UX-focused product design
- Real-world CRUD architecture

Main Takeaway

A polished user experience and reliable workflow are often more valuable than excessive technical complexity.

Future Improvements

- Email notifications
- Real-time updates
- Image-based AI matching
- QR-code based recovery
- Moderation system
- Push notifications
- Advanced analytics dashboard

Thank You

Questions?